Intelli Service API Documentation

1. Overview

This document describes the API endpoints required to authenticate a user with OTP and retrieve the user's subscription status for an Intelli service.

The API flow is typically:

1. Send OTP to the user's MSISDN.
2. Verify the OTP submitted by the user.
3. Fetch the user's subscription status.

2. Base URL

```
https://api.intellihq.net
```

3. Service Context

The current service identifier in the provided URLs is:

```
60
```

All endpoints are scoped under:

```
/api/v1/service/60/
```

4. Data Format

All API requests and responses use JSON.

Required Headers

```
Content-Type: application/json
Accept: application/json
```

5. MSISDN Format

The `msisdn` field represents the user's phone number.

Recommended format:

```
2348012345678
```

That is, the MSISDN should include the country code and should not include the leading `0`, `+`, spaces, or special characters.

Example:

```
{
  "msisdn": "2348012345678"
}
```

6. Endpoint Summary

Endpoint	Method	Purpose
`/api/v1/service/60/auth/send-otp/`	POST	Send OTP to user
`/api/v1/service/60/auth/verify-otp/`	POST	Verify user OTP
`/api/v1/service/60/subscription/status/`	GET	Fetch user subscription status

---

7. Send OTP to User

Description

This endpoint sends an OTP to the user's MSISDN. It is used as the first step in verifying the user's identity.

Endpoint

```
POST /api/v1/service/60/auth/send-otp/
```

Full URL

```
https://api.intellihq.net/api/v1/service/60/auth/send-otp/
```

Request Payload

```
{
  "msisdn": "string",
  "telco": "MTN"
}
```

Request Fields

Field	Type	Required	Description
`msisdn`	string	Yes	User's phone number in international format, for example `2348012345678`.
`telco`	string	Yes	User's telecom provider. Example: `MTN`.

Example Request

```
curl -X POST "https://api.intellihq.net/api/v1/service/60/auth/send-otp/"
  -H "Content-Type: application/json"
  -H "Accept: application/json"
  -d '{
    "msisdn": "2348012345678",
    "telco": "MTN"
  }'
```

Success Response

HTTP Status

```
200 OK
```

Response Body

```
{
  "success": true,
  "message": "OTP sent successfully to 2348012345678"
}
```

Response Fields

Field	Type	Description
`success`	boolean	Indicates whether the request was successful.
`message`	string	Human-readable success message.

Possible Error Responses

Invalid MSISDN or Missing Required Field

```
400 Bad Request
```

```
{
  "success": false,
  "message": "Invalid or missing msisdn"
}
```

Unsupported Telco

```
400 Bad Request
```

```
{
  "success": false,
  "message": "Invalid or unsupported telco"
}
```

---

8. Verify OTP

Description

This endpoint verifies the OTP submitted by the user. Once verification is successful, the API returns the user's subscription information.

Endpoint

```
POST /api/v1/service/60/auth/verify-otp/
```

Full URL

```
https://api.intellihq.net/api/v1/service/60/auth/verify-otp/
```

Request Payload

```
{
  "msisdn": "string",
  "otp": "string"
}
```

Request Fields

Field	Type	Required	Description
`msisdn`	string	Yes	User's phone number in international format.
`otp`	string	Yes	OTP received by the user.

Example Request

```
curl -X POST "https://api.intellihq.net/api/v1/service/60/auth/verify-otp/"
  -H "Content-Type: application/json"
  -H "Accept: application/json"
  -d '{
    "msisdn": "2348012345678",
    "otp": "123456"
  }'
```

Success Response

HTTP Status

```
200 OK
```

Response Body

```
{
  "success": true,
  "message": "OTP verified successfully",
```

```
  "data": {
    "service_id": 42,
    "msisdn": "2348012345678",
    "has_any_subscription": true,
    "has_active_subscription": true,
    "active_subscription": {
      "subscription_id": 12345,
      "sub_status": "active",
      "sub_active": true,
      "telco": "MTN",
      "traffic_source": "MOBPLUS",
      "active_product_id": 10,
      "auto_renewal": true,
      "starts_date": "2025-11-20T10:00:00+01:00",
      "ends_date": "2025-11-27T10:00:00+01:00"
    }
  }
}
```

Success Response Fields

Field	Type	Description
`success`	boolean	Indicates whether OTP verification was successful.
`message`	string	Human-readable success message.
`data`	object	User and subscription details.
`data.service_id`	integer	ID of the service.
`data.msisdn`	string	User's phone number.
`data.has_any_subscription`	boolean	Indicates whether the user has ever had a subscription.
`data.has_active_subscription`	boolean	Indicates whether the user currently has an active subscription.
`data.active_subscription`	object/ null	Active subscription details, if available.

Active Subscription Fields

Field	Type	Description
`subscription_id`	integer	Unique ID of the subscription record.

Field	Type	Description
`sub_status`	string	Subscription status. Example: `active`.
`sub_active`	boolean	Indicates whether the subscription is active.
`telco`	string	Telco associated with the subscription.
`traffic_source`	string	Source through which the subscription was acquired.
`active_product_id`	integer	Active product ID attached to the subscription.
`auto_renewal`	boolean	Indicates whether the subscription renews automatically.
`starts_date`	string	Subscription start date in ISO 8601 format.
`ends_date`	string	Subscription end date in ISO 8601 format.

Possible Error Responses

Invalid OTP

```
400 Bad Request
```

```
{
  "success": false,
  "message": "Invalid OTP"
}
```

Expired OTP

```
400 Bad Request
```

```
{
  "success": false,
  "message": "OTP has expired"
}
```

Missing OTP

```
400 Bad Request
```

```
{
  "success": false,
  "message": "OTP is required"
}
```

---

9. Fetch User Subscription Status

Description

This endpoint retrieves the subscription status of a user using their MSISDN.

It can be used after OTP verification or at any point where the client needs to confirm whether the user has an active subscription.

Endpoint

```
GET /api/v1/service/60/subscription/status/?msisdn={msisdn}
```

Full URL

```
https://api.intellihq.net/api/v1/service/60/subscription/status/?
msisdn={user_msisdn}
```

Query Parameters

Parameter	Type	Required	Description
`msisdn`	string	Yes	User's phone number in international format.

Example Request

```
curl -X GET "https://api.intellihq.net/api/v1/service/60/subscription/status/?
msisdn=2348012345678"
  -H "Accept: application/json"
```

9.1 Active Subscription Response

HTTP Status

```
200 OK
```

Response Body


```
{
  "success": true,
  "data": {
    "service_id": 42,
    "msisdn": "2348012345678",
    "has_any_subscription": true,
    "has_active_subscription": true,
    "active_subscription": {
      "subscription_id": 12345,
      "sub_status": "active",
      "sub_active": true,
      "telco": "MTN",
      "traffic_source": "MOBPLUS",
      "active_product_id": 10,
      "auto_renewal": true,
      "starts_date": "2025-11-20T10:00:00+01:00",
      "ends_date": "2025-11-27T10:00:00+01:00"
    }
  }
}
```

9.2 No Active Subscription Response

HTTP Status

```
200 OK
```

Response Body


```
{
  "success": true,
  "data": {
    "service_id": 42,
```

```
    "msisdn": "2348012345678",
    "has_any_subscription": false,
    "has_active_subscription": false,
    "active_subscription": null,
    "billing_records": [],
    "client_action": {
      "action": "redirect",
      "redirection_url": "https://example.com/subscribe"
    }
  }
}
```

Response Fields

Field	Type	Description
`success`	boolean	Indicates whether the request was processed successfully.
`data`	object	Subscription status data.
`data.service_id`	integer	ID of the service.
`data.msisdn`	string	User's phone number.
`data.has_any_subscription`	boolean	Indicates whether the user has ever had a subscription.
`data.has_active_subscription`	boolean	Indicates whether the user currently has an active subscription.
`data.active_subscription`	object/ null	Active subscription details if available; otherwise `null`.
`data.billing_records`	array	Billing records related to the user, if available.
`data.client_action`	object	Recommended action for the client application.
`data.client_action.action`	string	Action the client should perform. Example: `redirect`.
`data.client_action.redirection_url`	string	URL where the client should redirect the user.

Possible Error Responses

Missing MSISDN

```
400 Bad Request
```

```
{
  "success": false,
  "message": "MSISDN is required"
}
```

Invalid MSISDN

```
400 Bad Request
```

```
{
  "success": false,
  "message": "Invalid MSISDN"
}
```

---

10. Recommended Client Flow

Step 1: Collect User MSISDN and Telco

The client application should collect the user's phone number and telco.

Example:

```
{
  "msisdn": "2348012345678",
  "telco": "MTN"
}
```

Step 2: Send OTP

Call:

```
POST /api/v1/service/60/auth/send-otp/
```

If successful, prompt the user to enter the OTP received.

Step 3: Verify OTP

Call:

```
POST /api/v1/service/60/auth/verify-otp/
```

If verification is successful, inspect:

```
{
  "has_active_subscription": true
}
```

Step 4: Route User Based on Subscription Status

If User Has Active Subscription

Allow the user to access the service.

```
{
  "has_active_subscription": true,
  "active_subscription": {
    "sub_status": "active"
  }
}
```

If User Has No Active Subscription

Use the `client_action` object, if present.

Example:

```
{
  "client_action": {
    "action": "redirect",
    "redirection_url": "https://example.com/subscribe"
```

```
  }
}
```

The client should redirect the user to the provided `redirection_url`.

---

11. JavaScript Integration Examples

11.1 Send OTP

```
async function sendOtp(msisdn, telco) {
  const response = await fetch("https://api.intellihq.net/api/v1/service/60/
auth/send-otp/", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Accept": "application/json"
    },
    body: JSON.stringify({
      msisdn,
      telco
    })
  });

  const data = await response.json();

  if (!response.ok || !data.success) {
    throw new Error(data.message || "Unable to send OTP");
  }

  return data;
}
```

11.2 Verify OTP

```
async function verifyOtp(msisdn, otp) {
  const response = await fetch("https://api.intellihq.net/api/v1/service/60/
auth/verify-otp/", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Accept": "application/json"
    },
```

```
    body: JSON.stringify({
      msisdn,
      otp
    })
  });

  const data = await response.json();

  if (!response.ok || !data.success) {
    throw new Error(data.message || "Unable to verify OTP");
  }

  return data;
}
```

11.3 Fetch Subscription Status

```
async function fetchSubscriptionStatus(msisdn) {
  const url = new URL("https://api.intellihq.net/api/v1/service/60/subscription/
status/");
  url.searchParams.set("msisdn", msisdn);

  const response = await fetch(url.toString(), {
    method: "GET",
    headers: {
      "Accept": "application/json"
    }
  });

  const data = await response.json();

  if (!response.ok || !data.success) {
    throw new Error(data.message || "Unable to fetch subscription status");
  }

  return data;
}
```

11.4 Example Client Routing Logic

```
async function handleUserAccess(msisdn) {
  const subscriptionStatus = await fetchSubscriptionStatus(msisdn);
  const statusData = subscriptionStatus.data;
```

```
  if (statusData.has_active_subscription) {
    return {
      action: "grant_access",
      subscription: statusData.active_subscription
    };
  }

  if (statusData.client_action?.action === "redirect") {
    window.location.href = statusData.client_action.redirection_url;
    return;
  }

  return {
    action: "deny_access",
    message: "No active subscription found"
  };
}
```

---

12. Status Code Guide

Status Code	Meaning	Description
`200`	OK	Request was successful.
`400`	Bad Request	Invalid request payload, missing field, invalid OTP, or expired OTP.
`404`	Not Found	Endpoint or resource not found.
`405`	Method Not Allowed	Wrong HTTP method used for the endpoint.
`500`	Internal Server Error	Unexpected server error.

---

13. Error Handling Recommendation

Client applications should not rely only on the HTTP status code. They should also check the `success` field in the response body.

Recommended pattern:

```
if (!response.ok || data.success === false) {
  showError(data.message || "Something went wrong. Please try again.");
}
```

For OTP errors, the client should display a clear message and allow the user to request a new OTP.

Example user-facing messages:

API Error	Suggested Client Message
`Invalid OTP`	The OTP you entered is incorrect. Please check and try again.
`OTP has expired`	This OTP has expired. Please request a new OTP.
`MSISDN is required`	Please enter your phone number.
`Invalid MSISDN`	Please enter a valid phone number.

---

14. Security Recommendations

1. OTP should have an expiry window.
2. OTP verification attempts should be rate-limited.
3. MSISDN should be normalized before sending to the API.
4. Client applications should not expose sensitive logs containing OTP values.
5. HTTPS must always be used.
6. OTP should never be stored in local storage or persistent browser storage.
7. The client should handle failed verification attempts gracefully.

---

15. Notes for Frontend Developers

- Always normalize the user's phone number before making API calls.
- Always check both `response.ok` and the response body's `success` field.
- For users without active subscriptions, check whether `client_action` exists.
- If `client_action.action` is `redirect`, redirect the user to `client_action.redirection_url`.
- Do not hardcode the subscription URL if the API already returns `redirection_url`.
- Treat `active_subscription` as nullable.
- Use loading states during OTP sending and verification.
- Prevent users from submitting OTP multiple times rapidly.

---

16. Example End-to-End Flow

```
async function completeOtpLoginFlow(msisdn, telco, otp) {
  await sendOtp(msisdn, telco);

  const verificationResult = await verifyOtp(msisdn, otp);
  const userData = verificationResult.data;

  if (userData.has_active_subscription) {
    return {
      authenticated: true,
      subscribed: true,
      subscription: userData.active_subscription
    };
  }

  const subscriptionStatus = await fetchSubscriptionStatus(msisdn);

  return {
    authenticated: true,
    subscribed: subscriptionStatus.data.has_active_subscription,
    action: subscriptionStatus.data.client_action || null
  };
}
```

---

17. Changelog

Version	Date	Description
`1.0`	2026-05-12	Initial API documentation for OTP authentication and subscription status endpoints.